Robust Training of Neural Networks at Arbitrary Precision and Sparsity

C. Ye, G. Chu, Y. Liu, Y. Zhang, L. Lew, L. Zhang, M. Sandler, A. Howard
Google DeepMind • ICLR 2026

Presented by

Ayush Srivastava (2026AIB2015) Sarvesh Nikas (2026AIB2045)

Quantization and the forward pass during training

Why. A language model has billions of weights, each a 16- or 32-bit float: too big for a phone, slow, and costly in energy. Quantization stores each number with k bits ($k = 4, 2, 1$). Notation AaWw: a-bit activations, w-bit weights. At A1W1 a matrix multiply becomes XNOR and bit-counting.

Training (QAT). Keep the float block $x \in \mathbb{R}^N$ and update it by gradient descent, but run the forward pass on its quantized version, so the network learns to live with rounding. After training only the integers q and one scale and offset per block are kept.

$$x \xrightarrow{f} f(x) \xrightarrow{\text{round}} q \xrightarrow{\text{dequantize}} \hat{x} \longrightarrow \text{rest of network} \longrightarrow L$$

- $f(x) = \dfrac{x - b_f \mathbf{1}}{s_f}$ rescales x onto the grid $\{0, \ldots, 2^k - 1\}$, with $s_f = \dfrac{x_{\max} - x_{\min}}{2^k - 1}$, $b_f = x_{\min}$ (affine; linear has $b_f = 0$).
- $q = \text{round}\big(f(x)\big) = f(x) + \delta$: the stored integers, with rounding error $|\delta_i| \le \frac{1}{2}$.
- Standard dequantization: $\hat{x} = s_f\, q + b_f \mathbf{1} = x + s_f\, \delta$. This is what the next layer uses.

Example ($k = 2$). $x = (0.9, -0.3, 0.1, 0.6)$: $s_f = 0.4$, $b_f = -0.3$, $f(x) = (3, 0, 1, 2.25)$, $q = (3, 0, 1, 2)$, $\hat{x} = (0.9, -0.3, 0.1, \mathbf{0.5})$.

The problem: rounding has no gradient, and STE patches it blindly

Zero gradient. By the chain rule the scalings cancel:

$$\frac{\partial \hat{x}}{\partial x} = s_f \cdot \frac{\partial q}{\partial f(x)} \cdot \frac{1}{s_f} = \text{round}'\big(f(x)\big) = 0 \quad \text{almost everywhere,}$$

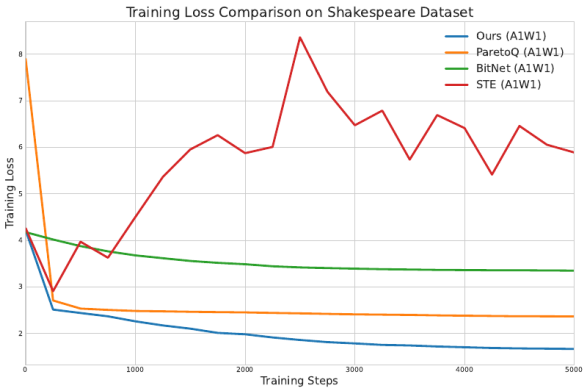
so $\nabla_x L = 0$: x and every earlier layer never update.

Straight-Through Estimator (Bengio et al., 2013). Round in the forward pass, pretend it is the identity in the backward pass:

$$q = f(x) + \delta, \qquad \delta = \text{stopgrad}\big(\text{round}(f(x)) - f(x)\big).$$

$$\text{forward:} \quad \hat{x} = x + s_f\,\delta, \qquad \text{backward:} \quad \frac{\partial \hat{x}}{\partial x} = I^{*}.$$

The blind spot. The forward pass shifts every entry by $s_f \delta_i$, but the backward pass is the same as with no quantization: δ never reaches the gradient (the paper's "ghost of departed quantities"). Earlier layers get no signal about how rounding hurts the loss. Harmless at 8 bits; at 1–2 bits $s_f \delta$ is as large as the weights and training becomes unstable.



Paper Fig. 1a: training loss at A1W1, small GPT on Shakespeare. STE (red) diverges; the paper's method (blue) converges.

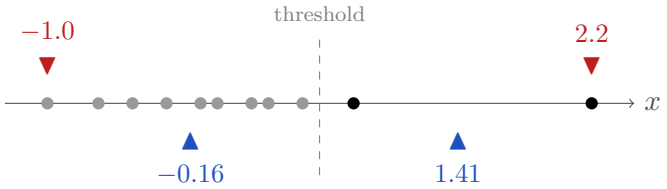
*Treating s_f, b_f as constants; they depend on x only through $x_{\min}, x_{\max}$.

Main idea: dequantize by ridge regression

Keep f and round. Replace $\hat{x} = s_f q + b_f \mathbf{1}$ by the best affine fit of x from q:

$$\min_{s_g, b_g} \frac{1}{2N} \left\| s_g\, q + b_g \mathbf{1} - x \right\|^2 + \frac{\lambda}{2} s_g^2 \quad \Longrightarrow \quad g(q) = \underbrace{\frac{\mathrm{Cov}_{xq}}{\mathrm{Var}_q + \lambda}}_{s_g} (q - \bar{q}\mathbf{1}) + \bar{x}\mathbf{1}.$$

- s_g, b_g come from the block's statistics at every step: nothing to learn, nothing to tune.
- Never worse: with $\lambda = 0$, $g(q)$ is the orthogonal projection of x onto $\mathrm{span}\{\mathbf{1}, q\}$, and $\hat{x}$ lies in that span, so $\|x - g(q)\| \le \|x - \hat{x}\|$.
- Example of slide 2: $g(q) = (0.935, -0.285, 0.122, 0.528)$; squared error $0.010 \to 0.007$.
- λ gives stability ($\mathrm{Var}_q = 0$ would give 0/0; $\lambda = 0$ produces NaN) and denoising ($\lambda \to \infty$ gives $g \to \bar{x}\mathbf{1}$). One value, $\lambda = 0.01$, is used everywhere.
- Same form and cost as a LayerNorm: $\mathrm{Var}_q + \lambda$ plays the role of $\sigma^2 + \epsilon$.



1 bit, 11 weights with one outlier. Both methods give the same q (grey $\to$ 0, black $\to$ 1). STE (▼) maps q back to $x_{\min}, x_{\max}$; ridge (▲) to roughly the two group means. Squared error $9.98 \to 2.93$.

Why it works: what the backward pass sees

Honest starting point. Ridge still sends the gradient straight through round (δ is detached, exactly as in STE). What changes is the function around round.

A true gradient now exists. Inside a rounding cell q is constant, yet g still depends on x through $\bar{x}$ and Cov_{xq}. So the deployed map $x \mapsto g\big(\mathrm{round}(f(x)), x\big)$ has a true Jacobian P (the projection onto $\mathrm{span}\{\mathbf{1}, q\}$): it moves the levels. For STE the true Jacobian is 0.

Jacobian used in training (our derivation; it matches autograd to machine precision*):

$$J_{\mathrm{STE}} = \underbrace{0}_{\text{exact}} + \underbrace{I}_{\text{surrogate}}, \qquad J_{\mathrm{ridge}} = \underbrace{P}_{\text{exact}} + \underbrace{\rho\,(I-P)}_{\text{surrogate}} + \underbrace{c\,u\,r^{\top}}_{\text{error term}},$$

$$u = q - \bar{q}\mathbf{1}, \quad r = x - g(q), \quad \rho = \frac{s_g}{s_f}, \quad c = \frac{1}{s_f N(\mathrm{Var}_q + \lambda)}, \quad P = \frac{\mathbf{1}\mathbf{1}^{\top}}{N} + \frac{uu^{\top}}{N(\mathrm{Var}_q + \lambda)}.$$

1. **Exact where possible.** The surrogate is used only for moving entries across rounding boundaries.
2. **The surrogate is damped.** $\rho = s_g/s_f$ was 0.29, 0.87, 0.99 at $k = 1, 2, 4$ (Gaussian block, $N = 64$). The coarser the rounding, the less the guess is trusted; at 4 bits ridge $\approx$ STE.
3. **The error itself is in the gradient.** $\nabla_x L$ gains the term $c\,r\,(u^{\top}\nabla_g L)$, where $u^{\top}\nabla_g L = \partial L/\partial s_g$: when the loss wants a different scale, x moves along its own error r. The term vanishes when $r = 0$.

*For all entries except the block's min and max. The paper's own version (App. A.4): $\partial g/\partial q = s_g I + q\,(\nabla_q s_g)^{\top}$ depends on δ through q.

Extensions: sparsity, and a fast affine matrix multiply

Sparsity is quantization to zero.

- Prune first: $\delta_S = \text{threshold}(x) - x$ (e.g. keep the 2 largest of every 4), $x_S = x + \delta_S$.
- Then quantize: $q = f(x_S) + \delta_Q$.
- $g(q)$ is fitted to the dense float x, so both passes see the total error $\delta_S + \delta_Q$.
- 1-bit weights with 1:4 sparsity: (1 value bit + 2-bit position)$/4 = 0.75$ bits per weight, i.e. below 1 bit.
- Gemma3 1B at A4W1: 2:4 sparsity halves the compute cost and raises accuracy $0.4056 \to 0.4080$.

Theorem 1 (affine matmul shortcut). Dequantize the rows of $X \in \mathbb{R}^{M\times n}$ and the columns of $W \in \mathbb{R}^{n\times P}$ with g:

$$\tilde{X} = (s_X \mathbf{1}^\top) \odot (Q_X - \bar{q}_X \mathbf{1}^\top) + \bar{x}\mathbf{1}^\top, \qquad \tilde{W} = (Q_W - \mathbf{1}\bar{q}_W^\top) \odot (\mathbf{1} s_W^\top) + \mathbf{1}\bar{w}^\top.$$

Then

$$\boxed{\tilde{X}\tilde{W} = (s_X s_W^\top) \odot \left(Q_X Q_W - n\,\bar{q}_X \bar{q}_W^\top\right) + n\,\bar{x}\,\bar{w}^\top}$$

Proof. Centred rows and columns sum to zero, so both cross terms vanish, and $\mathbf{1}^\top\mathbf{1} = n$. Expanding the centred integer product gives $Q_X Q_W - n\,\bar{q}_X \bar{q}_W^\top$. $\square$

Cost. One integer matmul $O(MnP)$, two rank-1 terms $O(MP)$, statistics $O(Mn + nP)$: about the cost of linear quantization. This makes affine quantization affordable. We checked the identity numerically (error 3×10^{-14}).

Results from the paper

Gemma3 1B, C4 accuracy	STE	ridge
A4W4, affine, SCQ128	0.4254	0.4274
A2W2, affine, SCQ128	0.3794	0.4027
A1.5W1.5, linear, channel	NaN	0.3297
A1W1, linear, SCQ128	0.3399	0.3547
A1W1, affine, SCQ128	0.3397	0.3751

SCQ128: one scale and offset per block of 128. The gap grows as bits fall. Affine gains nothing under STE ($0.3399 \rightarrow 0.3397$) but $+0.020$ under ridge.

- **Stability.** nanoGPT on Shakespeare at A1W1: STE and BitNet diverge or stall; ridge converges (slide 3). This also holds with linear quantization for both: STE ends above 5.0 validation loss, ridge at about 2.1. On GPT-2 124M (OpenWebText), STE and BitNet are unstable or reach NaN.
- **Storage frontier.** Best accuracy per weight bit comes from asymmetric A4W1, not A2W2: keep activations precise, compress static weights.
- **Bigger and fewer bits wins.** Gemma3 4B at A4W1 with 2:4 sparsity (650 MB) reaches 0.4517, above Gemma3 1B in BF16 (1400 MB, 0.4494).
- **Beyond LLMs, no tuning.** ResNet-50 on ImageNet: A4W4 76.45% vs. float 76.41%. WMT EN-DE: A4W4 29.71 vs. float 29.49 BLEU.

Our implementation: a toy reproduction

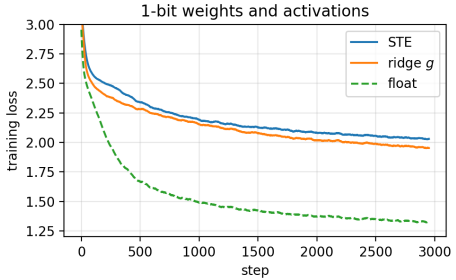
Setup. Char-level GPT (4 blocks, 0.82M parameters) on Tiny Shakespeare, 3000 steps, AdamW, one seed. All linear layers in the blocks at AkWk, affine, per row. Runs differ only in the dequantizer.

Core code (`quant.py`, condensed):

```
def quantize(x, k):              # q = f(x) + stopgrad(delta)
    lo = x.amin(-1, keepdim=True)
    hi = x.amax(-1, keepdim=True)
    s_f = (hi - lo + 1e-8) / (2**k - 1)
    f_x = (x - lo) / s_f
    delta = (torch.round(f_x) - f_x).detach()
    return f_x + delta, s_f, lo

def x_hat(q, s_f, lo):           # STE: undo the rescale
    return s_f * q + lo

def g(q, x, lam=1e-2):           # ridge fit of x from q
    m = lambda t: t.mean(-1, keepdim=True)
    qc, xc = q - m(q), x - m(x)
    s_g = m(xc * qc) / (m(qc * qc) + lam)
    return s_g * qc + m(x)
```



bits	STE	ridge
1	2.123	2.071
2	1.812	1.790
4	1.592	1.597
float	1.586	

Final validation loss (right); training loss at A1W1 (left).

- Ridge is better at 1–2 bits and ties at 4 bits, as $\rho \to 1$ predicts.
- STE did not diverge at this scale (the paper used 11M parameters and 5000 steps).
- Gradient check: same q, δ changed by up to 0.106. The gradient reaching x changes by 0.0000 under STE and by up to 0.0144 under ridge.
- Size at 1 bit: 3.27 MB $\to$ 256 kB (12.7×). 47% of that stays float: embeddings, output layer, LayerNorms.

Advantages and disadvantages

Advantages

- **Drop-in.** It replaces one function (about five lines). No learned quantizer parameters; one $\lambda = 0.01$ everywhere; same hyperparameters as the BF16 baseline.
- **Stable where STE fails.** A1W1, A1.5W1.5 and sub-1-bit train; the gain grows as bits fall.
- **Better forward pass.** Reconstruction is never worse than standard dequantization ($\lambda = 0$), at the cost of a LayerNorm.
- **Affine pays off**, and Theorem 1 makes it cheap.
- **One framework** for quantization and sparsity, with storage and energy trade-off curves for real LLMs.

Disadvantages

- **Still a surrogate through round.** "Without gradient estimation" is overstated; the exact part has rank 2 per block.
- **Little gain at 4 bits** (+0.002), where most deployment happens today.
- **Not free at inference.** s_g, b_g per block of 128 add 0.125–0.25 bits per weight (8- or 16-bit values); activations need float $\bar{x}$ and Cov_{xq} at run time.
- **Efficiency is estimated, not measured.** Energy = sparsity $\times$ A-bits $\times$ W-bits $\times$ ops ignores data movement. All runs use simulated (fake) quantization; no speed is reported.
- **Thin evidence.** Single runs, no error bars. $\lambda = 0 \to$ NaN is a 0/0 in the forward pass, not proof of the gradient path. Small errors: the linear closed form misses a factor N on λ, and the text after Eq. 6 has a typo.

Our opinion and takeaway

The real contribution. It is not "no gradient estimation": the guess through round is still there. The contribution is to change what surrounds the guess. A least-squares dequantizer gives an output closer to x and a Jacobian that is exact on the levels, damps the guess, and carries the error r. It is simple, cheap and principled; we would use it for 1–2-bit training.

Convincing. The ablation is controlled: same surrogate, only g changes, gains in every setting, and the gap grows as bits fall. The linear-vs-linear run rules out "it is just affine". Our ρ analysis predicts the same pattern.

Less convincing.

- "For the first time": LSQ (Esser et al., 2019) already puts δ into the gradient of a learned step size, $\partial\hat{x}/\partial s = \text{round}(x/s) - x/s = \delta$. Ridge fits scale and offset in closed form instead of learning them.
- Theorem 1 is the mean-centring identity. It holds for any affine dequantizer, with that dequantizer's output mean in place of $\bar{x}$; we checked this numerically.
- "Theoretically grounded" but there is no convergence result; the support is empirical.

Next steps for us. A λ sweep and linear vs. affine in our toy; several seeds; the paper's 11M model to see STE diverge; real integer kernels for wall-clock time.

Takeaway: do not ignore the quantization error; fit it. A ridge-regression dequantizer makes the forward pass less noisy and the backward pass error-aware, and that is enough to make 1-bit training stable.